

Machine Learning and Agentic AI for Macroeconomic Research

3: Machine Learning for Quantitative Macroeconomists

Zhigang Feng

Spring 2026

Lecture Agenda

1. **Why ML for Economists?** — unstructured data, forecasting, solving macro models
2. **Pattern Recognition** — handwritten digits as a motivating illustration
3. **Neural Networks** — architecture, activation functions, universal approximation
4. **Forward Propagation** — computing outputs layer by layer
5. **Backpropagation & Gradient Descent** — learning from errors
6. **Training Process** — loss functions, overfitting, regularization
7. **Specialized Architectures** — CNNs, convexity- & monotonicity-preserving networks
8. **Lab Preview** — curve fitting with Lab5A and Lab5B

Why ML for Economists?

Harnessing the Power of Unstructured Data

- Economics has traditionally relied on structured, tabular data. However, a wealth of economic information is locked in unstructured formats: text, images, satellite imagery.
- ML algorithms, particularly deep learning, provide tools to systematically process and quantify this unstructured data into economic indicators.
- **Applications:**
 - Measuring economic sentiment and uncertainty from financial news and social media
 - Analyzing satellite images to track economic activity (nighttime lights $\approx$ GDP proxy, ships in ports $\approx$ trade flows)
 - Using image recognition on product pictures to understand non-standardized product features

Extracting Insights from Textual Data

- Natural Language Processing (NLP) offers powerful techniques to analyze large volumes of text — going beyond simple word counts.
- **Applications:**
 - Gauging dovish or hawkish stance of central bankers from public communications
 - Identifying key risk factors from corporate annual reports
 - Quantifying the evolution of economic policy uncertainty from news articles
- **For macro:** FOMC minutes, Beige Books, and congressional records contain rich information that traditional econometrics cannot easily exploit.

Enhancing Causal Inference and Forecasting

- **Causal Inference:**

- “Double Machine Learning” allows flexible inclusion of high-dimensional controls, reducing omitted variable bias
- ML identifies *heterogeneous treatment effects*: how does a policy impact vary across subgroups?

- **Forecasting & Nowcasting:**

- ML captures complex, non-linear relationships that time-series models miss
- Regularization (e.g., LASSO) enables “kitchen sink” regression without overfitting
- Nowcasting: real-time economic indicators from credit card transactions, web searches, satellite imagery

Solving Large-Scale Macro and Finance Models

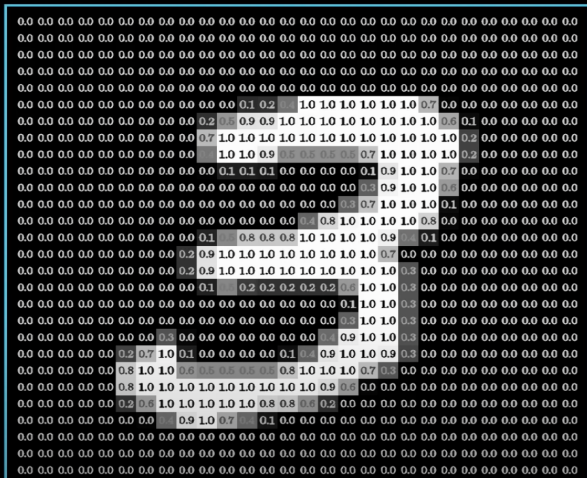
- Cutting-edge models (heterogeneous agents, portfolio choice) feature:
 - High dimensionality and complex non-linear dynamics
 - Traditional grid-based methods hit the **curse of dimensionality**
- ML techniques provide powerful new numerical methods:
 - **Deep learning:** parameterize policy/value functions with neural networks
 - **Reinforcement learning:** solve sequential decision problems via simulation
- **Examples:**
 - Optimal fiscal/monetary policy in heterogeneous agent models
 - Life-cycle portfolio choice with realistic income uncertainty
 - Krusell–Smith (1998) with neural network approximation of the law of motion

ML and the Curse of Dimensionality in Macro

- **CoD #1: Dimension of the state space**
 - $k \rightarrow \{k, a, h, \dots\}$; the distribution Γ is infinite-dimensional
 - *Traditional*: sparse grids, local approximation — scale poorly
 - *ML*: neural networks are **grid-free**; recover equilibrium functions via Monte Carlo sampling
- **CoD #2: Unfolding of uncertainty**
 - $\mathbb{E}$ over future contingencies; the space for Γ is unknown
 - *Traditional*: discretize shocks, impose Markov structure
 - *ML*: focus on the ergodic distribution; direct simulation Monte Carlo
- **Key insight**: ML shifts the computational bottleneck from *grid construction* to *function representation* — a much more scalable paradigm.

Pattern Recognition as Motivating Illustration

Example: Handwritten Digit Recognition



Digit Recognition as a Mathematical Problem

- **Dataset:** $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ with images $\mathbf{x}_i \in [0, 1]^{784}$ and labels $y_i \in \{0, \dots, 9\}$
- **Input–Output Mapping:**

$$f_{\theta}: \mathbf{x} \mapsto \hat{\mathbf{p}} \in \Delta^9, \quad \hat{y} = \arg \max_k \hat{p}_k$$

where Δ^9 is the 10-dimensional probability simplex

- **Learning Objective (Empirical Risk + Regularization):**

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \underbrace{\mathcal{L}(f_{\theta}(\mathbf{x}_i), y_i)}_{\text{cross-entropy}} + \lambda \|\theta\|_p$$

- **Search Strategy:** Stochastic gradient descent & variants (Adam, RMSProp)

Constraints and Structure in Handwritten Digits

- **Anatomical constraints:** Muscles, tendons, and joints dictate possible strokes — leading to systematic patterns in how digits are formed.
- **Geometric properties:** Each digit has defining features — ‘0’ has a closed loop, ‘1’ a vertical stroke, ‘8’ two stacked loops.
- **Spatial relationships:** Relative positions, sizes, and orientations of components maintain digit identity even with individual variation.
- **Lesson for economists:** Just as economic theory constrains the space of plausible models, structural constraints on handwriting reduce the effective dimensionality of the recognition problem.
- ML algorithms can **learn** these constraints from data — data augmentation, regularization, and architecture choices all exploit this structure.

From 784 Pixels to a Low-Dimensional Manifold

- **Manifold Hypothesis:** Natural digit images occupy a smooth, d -dimensional manifold $\mathcal{M} \subset \mathbb{R}^{784}$ with $d \ll 784$.
- **Two-Stage Model:**

$$\mathbf{x} \xrightarrow{\pi: \mathbb{R}^{784} \rightarrow \mathbb{R}^d} \mathbf{z} \xrightarrow{g_\phi: \mathbb{R}^d \rightarrow \Delta^9} \hat{\mathbf{p}}, \quad \hat{y} = \arg \max_k \hat{p}_k$$

- π : *encoder* — flattens the manifold to a latent code $\mathbf{z} \in \mathbb{R}^d$
- g_ϕ : *classifier head* — 1–2 FC layers + softmax $\rightarrow$ class probabilities
- **Choices for π :**
 - **Linear:** PCA, SVD, random projections
 - **Non-linear:** Auto-encoder, t-SNE, UMAP, first layers of a CNN
- **Macro analogy:** Finding a low-dimensional state in a high-dimensional model is the same problem — sufficient statistics for prediction.

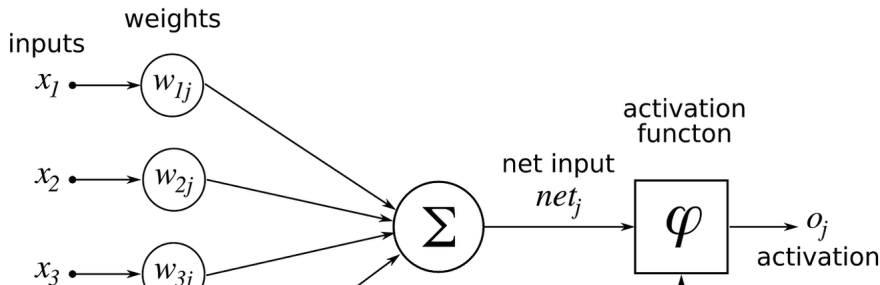
Defining a Deep Neural Network for Digit Recognition

- **Network Architecture:**
 - Input Layer: 784 neurons corresponding to pixel intensities
 - Hidden Layers: Multiple layers applying nonlinear transformations to learn complex patterns
 - Output Layer: 10 neurons, each representing one digit class (0–9)
- **Activation Functions:**
 - Nonlinear functions (ReLU, sigmoid) between layers
 - Customizable depth and width based on task complexity
- **Economic parallel:** The network structure mirrors the recursive structure in macro models — state variables are transformed layer by layer into decisions, just as $k_t \rightarrow c_t, l_t$ through policy functions.

Neural Networks: Architecture and Activation Functions

Neural Networks: The Basic Building Block

- Artificial neural networks are:
 - Inspired by biological neural networks
 - Composed of interconnected neurons organized in layers
 - Trained by adjusting weights based on data
- **Key components:**
 - **Neuron:** Receives inputs, applies an activation function, produces output
 - **Activation function:** Nonlinear map applied by each neuron (sigmoid, tanh, ReLU)
 - **Loss function:** Measures how well predictions match true values — the objective to minimize



Single Hidden Layer Network

- Input $x \in \mathbb{R}^M$, output $y \in \mathbb{R}^N$.
- The j -th hidden unit:

$$h_j^{(1)} = f_j^{(1)}(x) = \sigma \left(b_j^{(1)} + \sum_{m=1}^M w_{j,m}^{(1)} x_m \right), \quad j = 1, \dots, U_1 \quad (1)$$

- The i -th output:

$$y_i = \sum_{j=1}^{U_1} \tilde{w}_{i,j} h_j^{(1)} + \tilde{b}_i, \quad i = 1, \dots, N \quad (2)$$

- σ is a nonlinear activation function.
- Common choices: $\sigma(x) = \tanh(x)$, $\sigma(x) = \max(0, x)$ (ReLU), $\sigma(x) = \frac{1}{1+e^{-x}}$ (sigmoid).

Two Hidden Layer Network

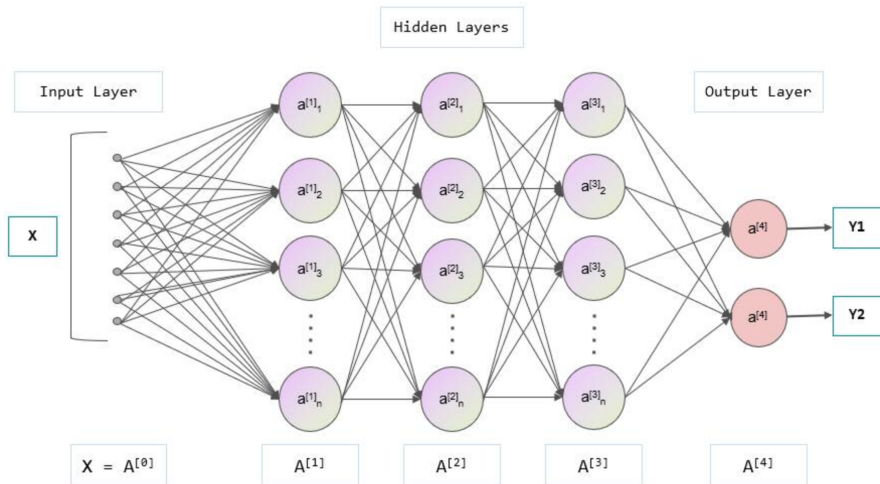
- Take the linear combination of $h^{(1)}$ and pass through another activation:

$$h_j^{(2)} = \sigma \left(b_j^{(2)} + \sum_{k=1}^{U_1} w_{j,k}^{(2)} f_k^{(1)}(x) \right), \quad j = 1, \dots, U_2 \quad (3)$$

$$y_i = \sum_{j=1}^{U_2} \tilde{w}_{i,j} h_j^{(2)} + \tilde{b}_i, \quad i = 1, \dots, N \quad (4)$$

- **Pattern:** Each layer performs a linear transformation followed by a nonlinear activation. Depth adds representational power.
- **Macro analogy:** Successive layers are like successive time steps in a dynamic program — each transforms the “state” into a richer representation.

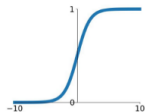
Hidden Layers: Visual



Activation Functions

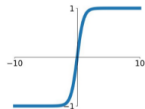
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



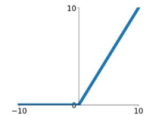
tanh

$$\tanh(x)$$



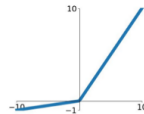
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

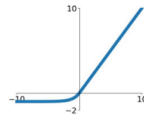


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



- **Sigmoid:** $\sigma(x) = \frac{1}{1+e^{-x}}$ — outputs in $(0, 1)$, used for probabilities
- **Tanh:** $\tanh(x)$ — outputs in $(-1, 1)$, zero-centered
- **ReLU:** $\max(0, x)$ — simple, sparse activation, dominant in modern networks

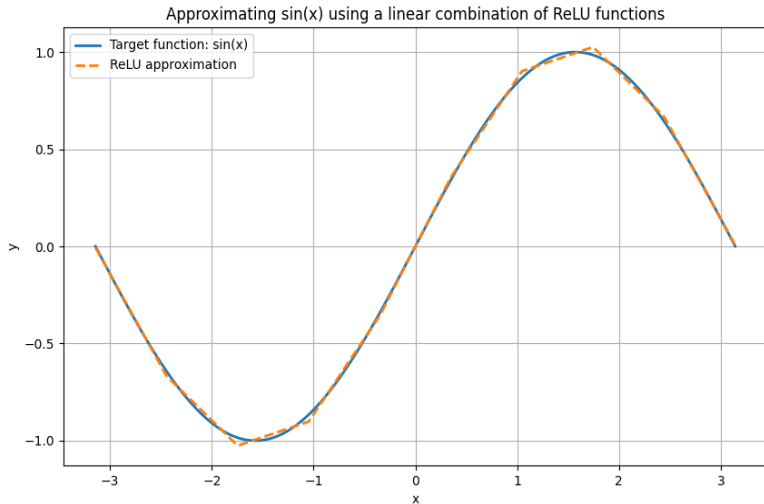
Function Approximation with ReLU

- **Goal:** Approximate arbitrary nonlinear functions using simple building blocks.
- **Key Idea:** Linear combination of shifted ReLU functions:

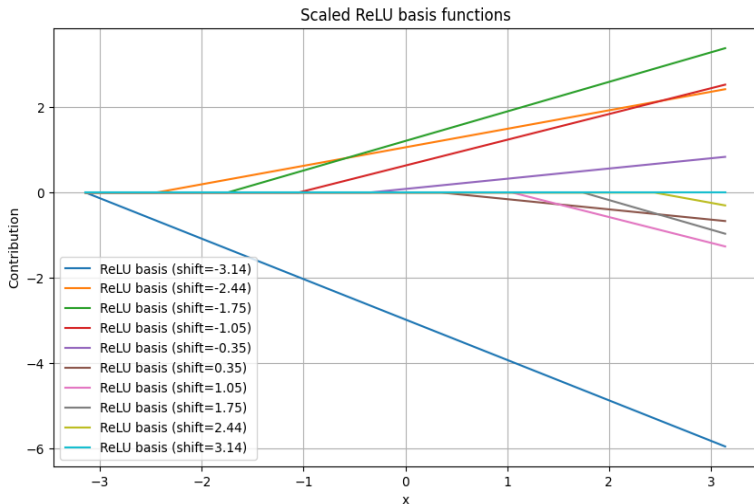
$$\hat{y}(x) = a_0 + \sum_{i=1}^N a_i \text{ReLU}(x - b_i)$$

- **Shifts** b_i : Define the “kink” points where marginal slope changes.
- **Coefficients** a_i : Scale each ReLU’s contribution to match local slope.
- **Example:** Approximating $\sin(x)$ on $[-\pi, \pi]$:
 - Choose shifts b_i evenly over the domain
 - Form design matrix with columns $\text{ReLU}(x - b_i)$ and a bias
 - Solve a least-squares problem for coefficients a_i
- **Economist’s take:** ReLU kinks are piecewise-linear splines — the network *learns* the knot locations, unlike a fixed Chebyshev grid.

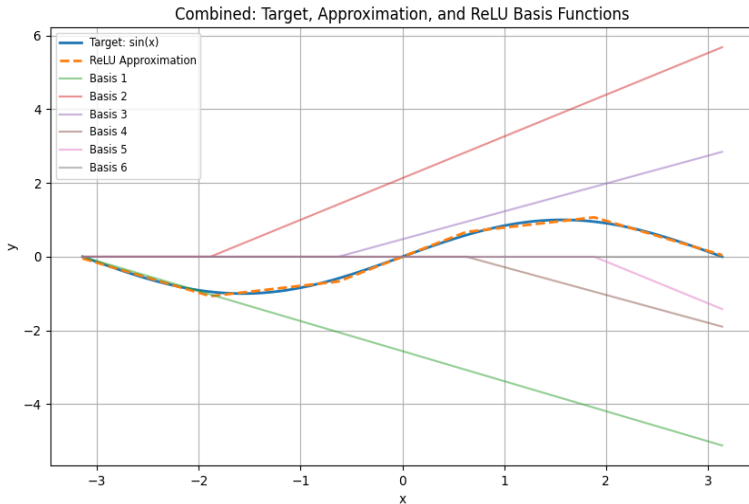
ReLU Approximation: Increasing Complexity



ReLU Approximation: More Kink Points



ReLU Approximation: High Fidelity



Why Neural Networks Work: Theoretical Guarantees

Universal Approximation Theorem (Cybenko, 1989)

If σ is sigmoidal (i.e., $\lim_{z \rightarrow -\infty} \sigma(z) = 0$, $\lim_{z \rightarrow +\infty} \sigma(z) = 1$), then any continuous function on $[0, 1]^d$ can be approximated uniformly by a two-layer neural network.

Breaking the Curse of Dimensionality (Barron, 1993)

A one-layer neural network achieves integrated squared error of order $O(1/M)$, where M is the number of nodes. In contrast, series approximations (polynomials, splines) achieve $O(1/M^{2/N})$ where N is the input dimension — exponentially worse.

- **For economists:** Barron's result explains why neural networks scale to high-dimensional state spaces where Chebyshev polynomials break down.

DNN = Linear Transformation + Nonlinear Activation

- **Linear Transformation:**

- Each neuron performs a weighted sum of inputs: projects data onto a new axis
- The set of transformations across a layer creates a *new representation*
- Enables: flexible feature representation, information compression, interaction capture

- **Nonlinear Activation:**

- Allows the network to capture nonlinearity and complex patterns
- Creates curved decision boundaries in the feature space
- **Essential** for the universal approximation property

- **Together:** Each layer applies an affine map $Wx + b$ then a pointwise nonlinearity $\sigma(\cdot)$. Stacking layers composes these maps:

$$f(x) = \sigma_L \circ A_L \circ \cdots \circ \sigma_1 \circ A_1(x)$$

where $A_\ell(x) = W_\ell x + b_\ell$.

Forward Propagation

Feedforward: Computing Outputs Layer by Layer

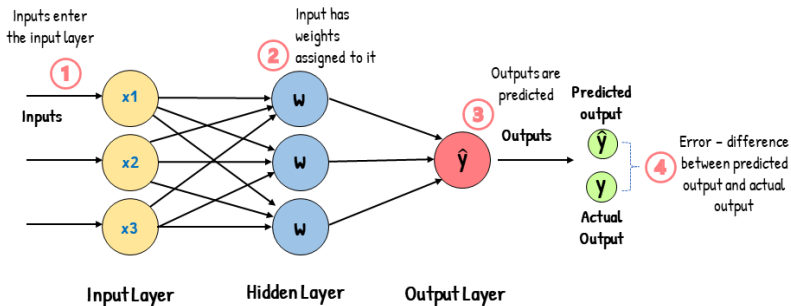
- **Forward propagation** (feedforward): compute the network output given an input by passing through layers sequentially.
- For a network with L hidden layers:

$$\mathbf{h}^{(0)} = \mathbf{x}, \quad \mathbf{h}^{(\ell)} = \sigma\left(W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}\right), \quad \ell = 1, \dots, L$$

$$\hat{\mathbf{y}} = W^{(L+1)}\mathbf{h}^{(L)} + \mathbf{b}^{(L+1)}$$

- Each layer: (1) multiply by weight matrix, (2) add bias, (3) apply activation.
- **Analogy:** Forward propagation is like solving a recursive model forward in time — given initial state x , compute successive transformations to arrive at the output.

Feed-Forward Neural Network



Toy Example: Forward Pass with Numbers

Consider a tiny network: input $x = 1.0$, one hidden neuron (sigmoid), one output neuron (sigmoid).

Parameters: $w_1 = 0.5$, $b_1 = 0$, $w_2 = 2.0$, $b_2 = 0$

Step 1: Hidden layer

$$z_1 = w_1 \cdot x + b_1 = 0.5 \quad \Rightarrow \quad h = \sigma(0.5) \approx 0.62$$

Step 2: Output layer

$$z_2 = w_2 \cdot h + b_2 = 2.0 \times 0.62 \approx 1.24 \quad \Rightarrow \quad \hat{y} = \sigma(1.24) \approx 0.78$$

Step 3: Loss (target $t = 0.0$)

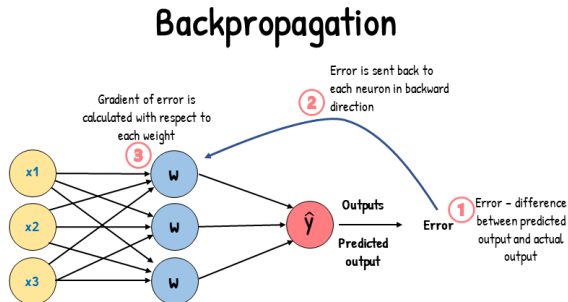
$$L = \frac{1}{2}(\hat{y} - t)^2 = \frac{1}{2}(0.78)^2 \approx 0.30$$

This is a complete forward pass: from x to $\hat{y}$ to loss L .

Backpropagation and Gradient Descent

Backpropagation: Learning from Errors

- **Goal:** Compute $\frac{\partial L}{\partial \theta}$ for every parameter θ in the network.
- **Method:** Apply the **chain rule** backwards through the network layers.
- **Steps:**
 1. Forward pass: compute all activations and the loss
 2. Backward pass: propagate error signal from output to input
 3. Compute gradient of loss w.r.t. each weight and bias
 4. Update parameters: $\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} L$



Toy Example: Backpropagation — Output Layer

Continuing our numerical example ($\hat{y} \approx 0.78$, $h \approx 0.62$, target $t = 0$):

Output layer gradients:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - t \approx 0.78$$

$$\frac{\partial \hat{y}}{\partial z_2} = \hat{y}(1 - \hat{y}) \approx 0.78 \times 0.22 \approx 0.17$$

$$\frac{\partial L}{\partial z_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \approx 0.78 \times 0.17 \approx 0.13$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_2} = 0.13 \times h \approx 0.13 \times 0.62 \approx 0.08$$

Interpretation: The gradient 0.08 tells us: to reduce the loss, decrease w_2 (the output overshoot the target of 0).

Toy Example: Backpropagation — Hidden Layer & Update

Hidden layer gradients:

$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial z_2} \cdot w_2 = 0.13 \times 2.0 \approx 0.26$$

$$\frac{\partial h}{\partial z_1} = h(1 - h) \approx 0.62 \times 0.38 \approx 0.24$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial h} \cdot \frac{\partial h}{\partial z_1} \cdot x = 0.26 \times 0.24 \times 1.0 \approx 0.06$$

Parameter update (learning rate $\eta = 0.1$):

$$w_2^{\text{new}} \approx 2.0 - 0.1 \times 0.08 = 1.992, \quad w_1^{\text{new}} \approx 0.5 - 0.1 \times 0.06 = 0.494$$

Key point: This is exactly what PyTorch's `.backward()` and the optimizer do — just at massive scale with millions of parameters.

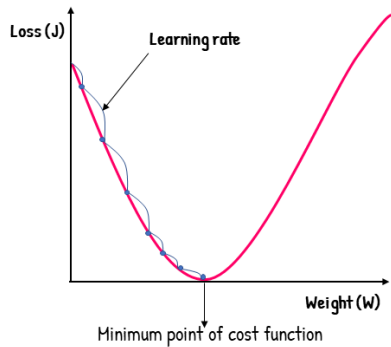
Gradient Descent: The Optimization Engine

- **Core update rule:** $\theta_{n+1} \leftarrow \theta_n - \lambda_n g(b_n; \theta_n)$
- **Variants by batch size n :**
 - $n = 1$: **Stochastic Gradient Descent (SGD)** — one random sample per step
 - $n = N$: **Full Gradient Descent (GD)** — all data per step
 - $1 < n < N$: **Mini-batch GD** — the practical sweet spot
- **Stochastic gradient approximation:**

$$\frac{d\mathcal{L}(\Gamma_\gamma)}{d\gamma} \approx \frac{1}{n} \sum_{i=1}^n \frac{d\mathcal{L}(\Gamma_\gamma; b_i)}{d\gamma}$$

- **Algorithm:**
 1. Initialize parameters randomly
 2. Draw random mini-batch b_n ; compute stochastic gradient $g(b_n; \theta_n)$
 3. Choose learning rate $\lambda_n > 0$; update $\theta_{n+1} \leftarrow \theta_n - \lambda_n g(b_n; \theta_n)$
 4. Repeat until convergence

Learning Rate



Adaptive Learning Rates: RMSProp and Adam

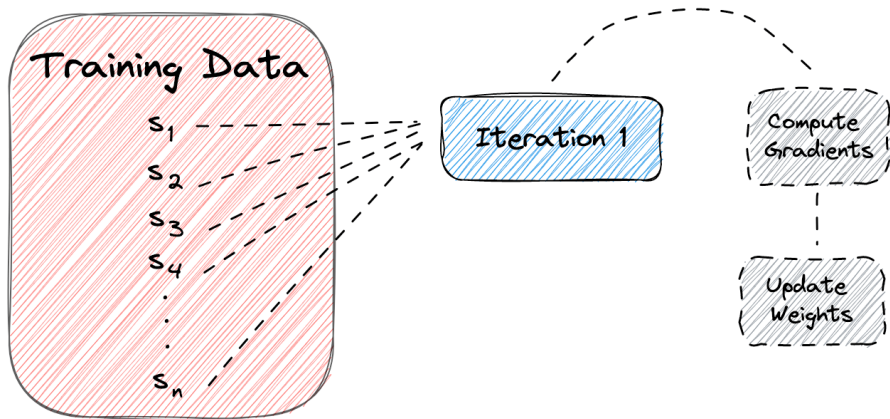
- **RMSProp**: Adjust learning rates individually by parameter:

$$g_k \leftarrow \frac{1}{n} \sum_{i=1}^n \nabla \mathcal{L}(\Gamma_\gamma; b_i)$$
$$r_{k+1} \leftarrow \rho r_k + (1 - \rho) g_k \odot g_k$$
$$\Delta \theta_{k+1} = - \frac{\lambda}{\sqrt{\delta + r_{k+1}}} \odot g_k$$

Divides by running average of squared gradients — normalizes step size.

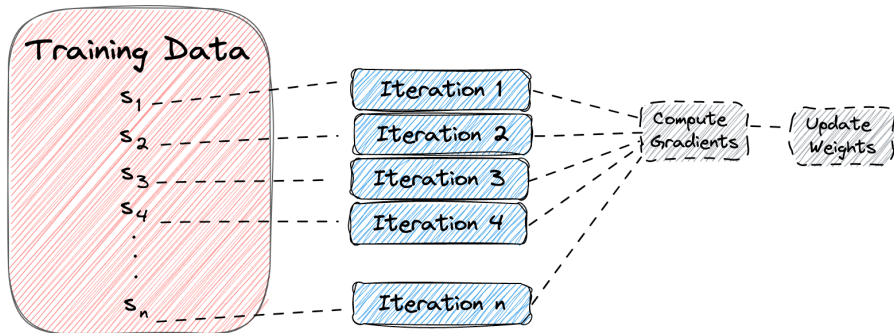
- **Adam** (Adaptive Moments): Combines momentum + RMSProp.
 - Tracks exponential moving average of gradient (1st moment) and squared gradient (2nd moment)
 - Parameters ρ_1, ρ_2 control decay rates
 - Default choice in most deep learning applications

Batch vs. Stochastic vs. Mini-Batch



Batch GD: Smooth convergence path, but each step is expensive (uses all data).

Stochastic Gradient Descent



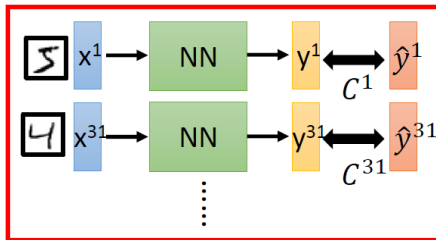
SGD: Noisy path with high variance, but each step is cheap and can escape local minima.

Mini-Batch Gradient Descent

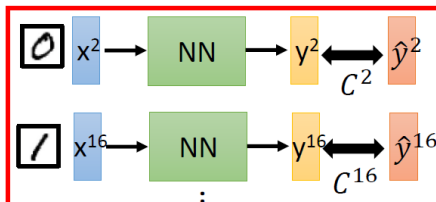
We do not really minimize total loss!

Mini-batch

Mini-batch



Mini-batch



- Randomly initialize network parameters
- Pick the 1st batch
 $L' = C^1 + C^{31} + \dots$
Update parameters once
- Pick the 2nd batch
 $L'' = C^2 + C^{16} + \dots$
Update parameters once
 $\vdots$
- Until all mini-batches have been picked

one epoch

Why Mini-Batch? Four Reasons

1. **Computational efficiency:** More frequent parameter updates than full GD; faster convergence in wall-clock time.
2. **Memory constraints:** Large datasets may not fit in GPU memory. Mini-batches process manageable subsets.
3. **Implicit regularization:** The stochasticity in gradient estimates helps escape local minima and saddle points — acts as a form of regularization.
4. **Parallelization:** Each mini-batch can be processed across multiple GPUs simultaneously.

Economist's note: Mini-batch SGD is the computational analogue of sampling-based estimation in econometrics — trade exactness for speed, and the noise is actually helpful.

Convergence Theory (For the Technically Inclined)

Convergence with Diminishing Learning Rate

Under standard assumptions (Lipschitz gradient, bounded variance), if the learning rate sequence satisfies

$$\sum_{n=1}^N \lambda_n = \infty, \quad \sum_{n=1}^N \lambda_n^2 < \infty$$

then the expected average squared gradient converges:

$$\mathbb{E} \left[\frac{1}{\Delta_N} \sum_{n=1}^N \lambda_n \|\nabla F(\theta_n)\|^2 \right] \xrightarrow{N \rightarrow \infty} 0$$

where $\Delta_N = \sum_{n=1}^N \lambda_n$.

— Bottou, Curtis, Nocedal (2018). *Optimization Methods for Large-Scale ML*.

- The conditions $\sum \lambda_n = \infty$ (keep exploring) and $\sum \lambda_n^2 < \infty$ (shrink steps) mirror Robbins–Monro conditions familiar from stochastic approximation in econometrics.

Training Process: Loss Functions, Overfitting, Regularization

Loss Functions

- **Mean Squared Error (regression):**

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

Equivalent to maximum likelihood under Gaussian errors.

- **Cross-Entropy (classification):**

$$\mathcal{L}_{\text{CE}} = - \sum_{k=0}^{K-1} y_k \log(\hat{y}_k)$$

where y_k is the one-hot ground truth and $\hat{y}_k = \text{softmax}(z)_k = \frac{\exp(z_k)}{\sum_j \exp(z_j)}$.

- **Intuition:** Cross-entropy is small when confident *and correct*, large when confident *but wrong*.
- **For economists:** MSE loss $\Leftrightarrow$ OLS objective. Cross-entropy $\Leftrightarrow$ multinomial logit likelihood.

Training Terminology

- **Epoch:** One complete pass through the entire training dataset.
- **Mini-batch:** A small subset of training data used for one gradient update.
- **Iteration:** One parameter update (= one mini-batch processed).
 - If dataset has 10,000 samples and batch size is 100, one epoch = 100 iterations.
- **Automatic Differentiation:** Computational technique for exact gradient computation — the engine behind `loss.backward()` in PyTorch.
- **Episode:** (Reinforcement learning context) One sequence of steps from initial state to terminal state — analogous to one simulated life-cycle path.

Overfitting and Stopping Criteria

- **The fundamental tension:**

- Training loss: always decreasing with more epochs
- Validation loss: decreases, then **increases** → overfitting

- **Stopping criteria:**

1. Fixed number of epochs (simple but suboptimal)
2. Monitor validation loss — stop when it stops improving
3. **Early stopping:** halt training when validation loss increases for k consecutive epochs

- **Model selection:** Choose the model (epoch checkpoint) with best validation performance.

- **Economist's analogy:** Overfitting is the ML version of data mining — the model memorizes noise in the training sample rather than learning the true data-generating process. Out-of-sample validation is the discipline.

Local Optima vs. Global Optimum

- **Non-convex loss landscape:**

- Neural network loss functions are *not* convex — many local optima and saddle points
- Global optimum exists but is generally unreachable in practice

- **Mitigation strategies:**

- Random initialization of weights (different starting points)
- Momentum and adaptive learning rates (escape shallow minima)
- Mini-batch stochasticity (natural exploration of the loss surface)

- **Practical reality:** Finding the global optimum is neither necessary nor desirable. A *good local optimum* that generalizes well is sufficient.

- **Research frontier:** “Loss landscape” analysis shows that in over-parameterized networks, most local minima achieve similar performance — the landscape is “benign” (Choromanska et al., 2015).

Regularization: Preventing Overfitting

- **L1 / L2 Regularization (weight decay):**

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{data}} + \lambda \|\theta\|_p, \quad p \in \{1, 2\}$$

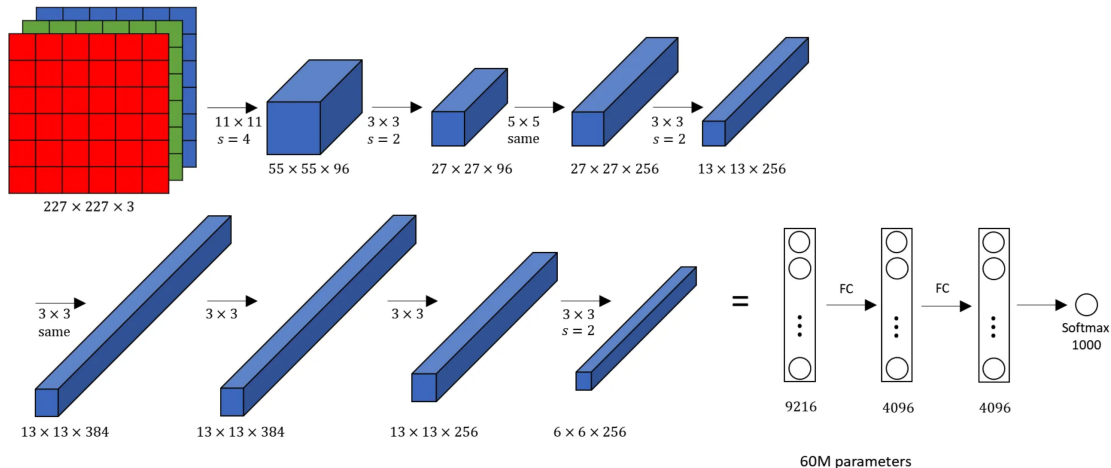
- L1 ($p = 1$): promotes sparsity (like LASSO in econometrics)
- L2 ($p = 2$): shrinks weights toward zero (like Ridge regression)
- **Dropout:** Randomly set a fraction of neurons to zero during each training step.
 - Forces the network to learn redundant representations
 - Equivalent to training an ensemble of sub-networks
- **Batch Normalization:** Normalize activations within each layer across the mini-batch. Stabilizes and accelerates training.
- **Data Augmentation:** Expand the training set with transformed copies (rotation, scaling, noise injection).

Specialized Architectures

Convolutional Neural Networks (CNNs): Learned Manifold Maps

- Recall the two-stage model: $\mathbf{x} \xrightarrow{\pi} \mathbf{z} \xrightarrow{g_\phi} \hat{\mathbf{p}}$
- In a CNN:
 - Stacked **convolution** + **nonlinearity** + **pooling** layers implement a *learned* π
 - Each filter detects a local pattern (edges, curves, textures)
 - **Convolution:** $Y(i, j) = \sum_m \sum_n X(i - m, j - n) W(m, n)$
 - **Pooling:** $\text{MaxPool}(i, j) = \max_{(m, n) \in R} X(m, n)$ — spatial summarization
- The fully connected “classifier head” is the g_ϕ map turning features into predictions.
- **Economic analogy:** CNN layers learn a *sufficient statistic* for prediction — like finding a low-dimensional state in a high-dimensional macro model.

AlexNet: A Landmark CNN Architecture



Krizhevsky, Sutskever, Hinton (2012). Won ImageNet with 15.3% top-5 error (vs. 26.2% for non-DL methods). This result launched the deep learning revolution.

Input Convex Neural Networks (ICNNs)

- **Motivation:** In economics, many functions are known to be convex or concave: cost functions, duality mappings, value functions (after sign change).
- **ICNN architecture** (Amos, Xu, Kolter, 2017): ensure $f(x; \theta)$ is convex in x by construction:

$$\mathbf{z}^{(0)} = x$$
$$\mathbf{z}^{(\ell+1)} = \sigma \left(\underbrace{W^{(\ell)}}_{\geq 0} \mathbf{z}^{(\ell)} + A^{(\ell)} x + b^{(\ell)} \right), \quad \ell = 1, \dots, L-1$$

- **Key constraints:**
 - Weights $W^{(\ell)} \geq 0$ (element-wise non-negative) for $\ell \geq 1$
 - Activation σ must be convex and non-decreasing (ReLU, softplus)
 - “Pass-through” connections $A^{(\ell)} x$ can have unconstrained weights
- **Result:** The composition of convex, non-decreasing functions with non-negative weights preserves convexity.
- **For concavity:** Use $g(x) = -f(x; \theta)$ where f is an ICNN.

Monotone Neural Networks

- **Motivation:** Economic theory often implies monotonicity:
 - Demand is non-increasing in price (law of demand)
 - Consumption is non-decreasing in wealth (normal goods)
 - Value functions are non-decreasing in the state
- **Architecture for monotonicity in input x_j :**
 - Constrain all weights connecting from x_j through the network to be **non-negative**
 - Use **non-decreasing** activation functions (ReLU, sigmoid, softplus)
 - Composition of non-decreasing functions is non-decreasing
- **Partial monotonicity:** Can enforce monotonicity w.r.t. *some* inputs while leaving others unconstrained — matching economic theory that demand is monotone in price but non-monotone in income (Giffen goods aside).
- **Practical enforcement:** After each gradient step, project weights to $\mathbb{R}_+$ via $w \leftarrow \max(w, 0)$ or use $w = \exp(\tilde{w})$ reparameterization.

Economic Applications of Constrained Architectures

- **ICNN for optimal transport / Wasserstein distance:**

- Kantorovich duality: $W_1(\mu, \nu) = \sup_{f \text{ 1-Lipschitz}} \mathbb{E}_\mu[f] - \mathbb{E}_\nu[f]$
- ICNNs parameterize the convex conjugate — used in generative models and distributional analysis

- **Concave value functions:**

- In many macro models, $V(k)$ is concave in capital
- Approximate V with $-\text{ICNN}(k)$ to enforce concavity *by construction*
- Eliminates spurious non-concavities from unconstrained approximation

- **Monotone policy functions:**

- Savings policy $a'(a, z)$ is typically increasing in current assets a
- Monotone networks guarantee this without post-hoc corrections

- **General principle:** Encoding economic structure into the network architecture reduces the hypothesis space, improves sample efficiency, and produces economically interpretable solutions.

Case Study: Curve Fitting and PyTorch

Case Study: DNN vs. Polynomial vs. Chebyshev

- **Objective:** Approximate a “kinked” function using three methods:
 - Deep Neural Network (DNN)
 - Traditional polynomial approximation (`np.polyfit`)
 - Chebyshev polynomial approximation
- **Why this matters for economists:**
 - Many economic models feature kinks: borrowing constraints, tax brackets, occasionally binding constraints
 - Polynomials suffer from **Gibbs phenomenon** near discontinuities
 - DNNs with ReLU activations handle kinks naturally (piecewise linear)
- **Test function:**

$$y(x) = \begin{cases} \alpha\beta x^\alpha & x \leq 0.5 \\ 2\alpha\beta x^\alpha & x > 0.5 \end{cases}$$

A power function with a discrete jump at $x = 0.5$ — like a policy threshold.

Example: GDP Forecasting with a Tiny MLP

- **Objective:** Predict next-quarter real GDP growth g_t from four lags.
- **Input:** $\mathbf{x}_t = (g_{t-1}, g_{t-2}, g_{t-3}, g_{t-4}) \in \mathbb{R}^4$
- **Architecture:**

$$\text{Input (4)} \xrightarrow{\text{ReLU}} \text{Hidden (16)} \xrightarrow{\text{ReLU}} \text{Hidden (8)} \xrightarrow{\text{Linear}} \text{Output (1)}$$

- **Why a neural network?**
 - Standard AR models assume linear persistence
 - GDP dynamics are often *state-dependent*: shocks have different persistence in recessions vs. expansions
 - The MLP allows coefficients to vary non-linearly with the lag structure
- **Loss function:** MSE $\equiv$ Gaussian MLE: $\mathcal{L}(\theta) = \frac{1}{N} \sum_t (\hat{g}_t - g_t)^2$

PyTorch: Defining and Training the Model

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 class GDPNet(nn.Module):
6     def __init__(self):
7         super().__init__()
8         self.net = nn.Sequential(
9             nn.Linear(4, 16), nn.ReLU(),
10            nn.Linear(16, 8), nn.ReLU(),
11            nn.Linear(8, 1)
12        )
13    def forward(self, x):
14        return self.net(x)
15
16 model = GDPNet()
17 criterion = nn.MSELoss()
18 optimizer = optim.Adam(model.parameters(), lr=0.001)
19
20 for epoch in range(1000):
21     optimizer.zero_grad()           # 1. Reset gradients
```

PyTorch: Class-Based Network with Batch Normalization

```
1 class KinkNet(nn.Module):
2     """Network for approximating kinked functions."""
3     def __init__(self, input_dim, hidden_dim1,
4                   hidden_dim2, output_dim=1):
5         super().__init__()
6         self.net = nn.Sequential(
7             nn.Linear(input_dim, hidden_dim1),
8             nn.BatchNorm1d(hidden_dim1),
9             nn.ReLU(),
10            nn.Linear(hidden_dim1, hidden_dim2),
11            nn.BatchNorm1d(hidden_dim2),
12            nn.ReLU(),
13            nn.Linear(hidden_dim2, output_dim)
14        )
15
16    def forward(self, x):
17        return self.net(x)
```

Key additions: BatchNorm1d normalizes activations within each mini-batch — stabilizes training and acts as light regularization.

Lab Preview

Lab: Machine Learning Basics and Applications

Lab 5A: Machine Learning Basics

Notebook: `Lab5A_MachineLearning_Basics.ipynb`

- Build and train neural networks from scratch in PyTorch
- Compare DNN, polynomial, and Chebyshev approximation on kinked functions
- Visualize the training process: loss curves, learned function shapes
- Experiment with architecture choices: depth, width, activation functions

Lab 5B: FOMC Minutes and 10-Year Yield Model

Notebook: `Lab5B_fomc_minutes_10y_model.ipynb`

- Apply ML to a real economic forecasting task
- Extract features from FOMC meeting minutes (text $\rightarrow$ numerical features)
- Train an MLP to predict 10-year Treasury yield movements
- Evaluate: out-of-sample performance, feature importance

Summary

1. **Why ML:** Unstructured data, high-dimensional models, and non-linear dynamics make ML essential for modern macro research.
2. **Neural Networks:** Universal approximators that compose linear transformations with nonlinear activations. Barron (1993): break the curse of dimensionality.
3. **Training:** Forward pass $\rightarrow$ loss $\rightarrow$ backpropagation $\rightarrow$ gradient descent. Mini-batch SGD with Adam is the practical standard.
4. **Regularization:** Weight decay, dropout, early stopping, batch normalization prevent overfitting — the ML analogue of avoiding data mining.
5. **Constrained Architectures:** ICNNs enforce convexity; monotone networks enforce monotonicity. Encoding economic theory into network design improves both accuracy and interpretability.
6. **Next:** Lec. 4 applies these tools to *solve* macro models — deep learning meets the Bellman equation.